UD 7

Control y manejo de Excepciones

–

Módulo de Programación
1º DAW

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Autor: Fran Gómez

¿Qué se va a evaluar?

	Mod.	RA
RA3 Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje	10%	
d) Se ha escrito código utilizando control de excepciones.	1%	10%

Índice de contenidos

1. Concepto
2. Jerarquía de excepciones
3. Miembros de una excepción
4. Manejo de excepciones
   4.1. Captura
   4.2. Propagación
   4.3. Lanzamiento
   4.4. Creación de excepciones personalizadas
5. Buenas prácticas
6. Aserciones

Excepciones: concepto

Evento producido durante la ejecución de un programa que altera el flujo normal de ejecución

```
public class Excepciones {
    public static void main(String[] args) {

        List lista = null;

        lista.add("A");

        System.out.println(lista);
    }
}
```

NullPointerException

Flujo normal
Flujo alternativo de la excepción

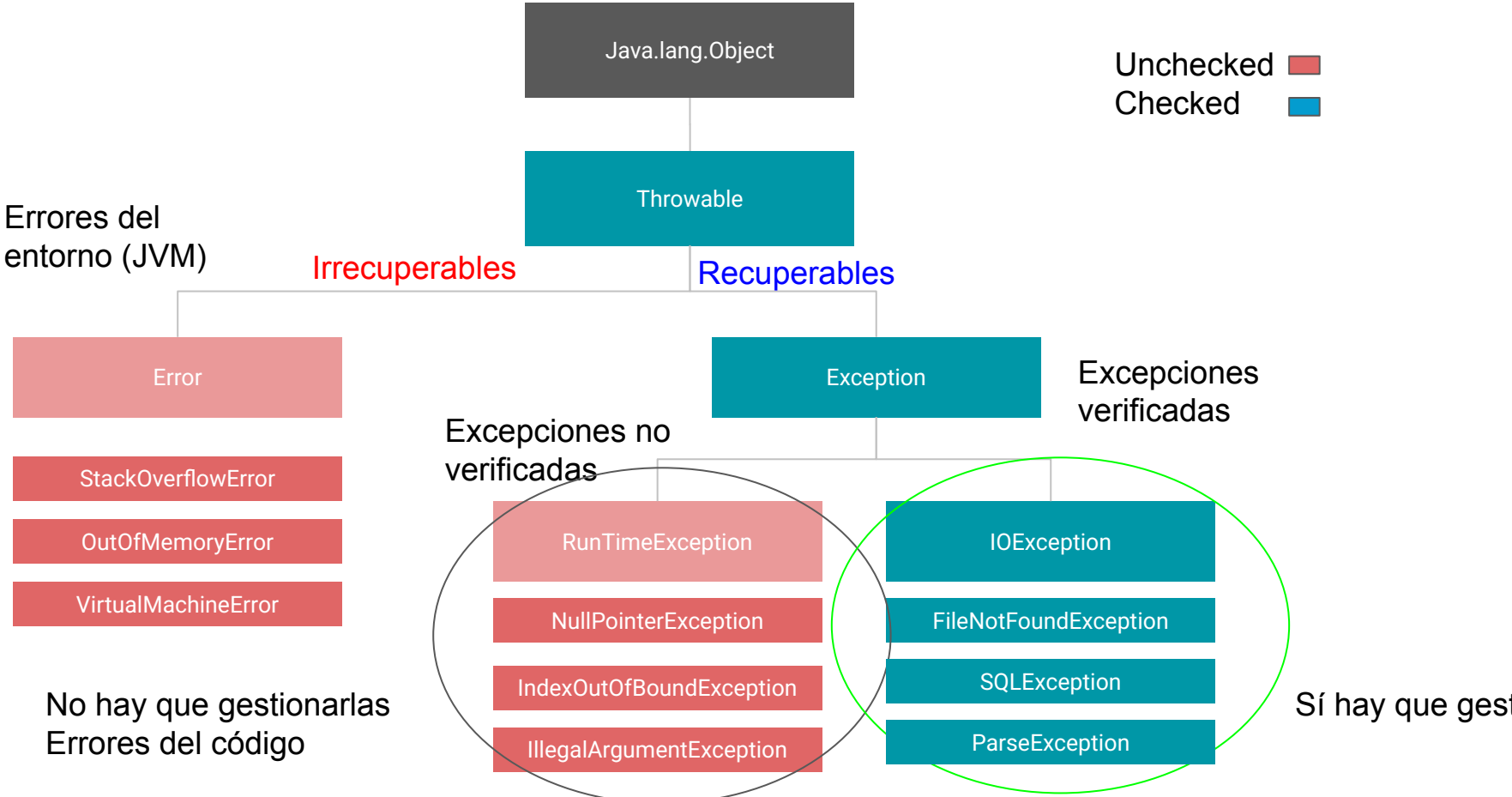
Excepciones: precedentes

Antes de que los lenguajes incorporasen la gestión de excepciones, tenía que hacerlo el programador: se podía hacer que el método devolviese un código de error y actuar en consecuencia.

```
public static void main(String[] args) {
    String codigoRespuesta = metodo("hola excepciones!");
    switch (codigoRespuesta){
        case "000": break; // Tratamiento para este error. Ej: volver a pea
        case "400": break; // ...
        case "500": break; // ...
        default:
            System.out.println("Finalizado correctamente.");
    }
}
```

```
static String metodo(String s) {
    if (s == null) {
        return "000";
    } else if (s.isEmpty()) {
        return "400";
    } else if (s.isBlank()) {
        return "500";
    }
    // Continúa el método
    // ...
    return "OK";
}
```

Excepciones: Jerarquía



Excepciones: clasificación

¿Prevenirlas o manejarlas?

Checked (verificadas o explícitas): Somos conscientes de que pueden ocurrir, pues dependen del uso del programa. El programa puede continuar pero obligatoriamente hay que indicarle el nuevo flujo que debe tomar.

Ej: Nuestro código lee de un fichero, pero ¿qué pasa si el usuario indica mal la ruta?→ *FileNotFoundException*

Unchecked (no verificadas o implícitas) : Son errores de programación que hemos cometido. El programa puede acabar inesperadamente, aunque podríamos definir un nuevo flujo, pero lo correcto es no hacerlo y corregir el error de programación. Java lanza estas excepciones automáticamente.

Ej: Accedemos a una posición fuera del array → *ArrayIndexOutOfBoundException*

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

Miembros de una excepción

Métodos constructores:

- Crear una excepción con un mensaje, con la causa o vacía

Métodos consultores:

- String **toString**: cadena que representa la excepción: *tipoExcepción: mensaje*
- Class<?> **getClass**: devuelve la clase a la que pertenece la excepción
- String **getMessage**: devuelve el mensaje
- **printStackTrace**: imprime traza del error

Manejo de excepciones

Cuando se lanza una excepción podemos:

- **Capturala** → try - catch - finally
- **Propagarla** → throws

Captura de excepciones

Sintaxis: bloque try - catch - finally

```
try {
    // código susceptible de generar excepciones
} catch (Tipo1 excepción1) {
    // código encargado de gestionar excepciones Tipo1
} catch (TipoN excepciónN) {
    // código encargado de gestionar excepciones TipoN
} finally {
    // código que siempre se ejecutará
}
```

Ejemplo


```java
try{
    FileReader fichero = new FileReader("nombre del fichero");
} catch (FileNotFoundException e) {
    e.printStackTrace();
}
```

¿Qué ocurre si el usuario introduce mal la ruta del fichero?

Ejemplo y Ejercicio 1


```java
System.out.println("COMIENZO PROGRAMA DIVISIÓN");
try {
    int dividendo = 5;
    int divisor = 0;
    System.out.println("Resultado :" + dividendo / divisor);
} catch (ArithmeticException ex){
    System.out.println("Imposible dividir por 0");
} finally {
    System.out.println("FIN PROGRAMA DIVISIÓN");
}
```

Propagación

Cuando se produce una excepción esta se puede pasar al código que invocó el método donde se produjo la excepción, para que sea este código el que la capture y la maneje.

Sintaxis:

[modificadores] tipo nombre ([args]) **throws TipoExcepción1, TipoExcepciónN**…

Ejemplo


```java
public static void main(String[] args) {

    System.out.println("COMIENZO PROGRAMA DIVISIÓN");
    try {
        int dividendo = 5;
        int divisor = 0;
        System.out.println("Resultado :" + division(dividendo, divisor));
    } catch (ArithmeticException ex){
        System.out.println("Imposible dividir por 0");
    } finally {
        System.out.println("FIN PROGRAMA DIVISIÓN");
    }

}

private static int division(int dividendo, int divisor) throws ArithmeticException {
    return dividendo / divisor;
}
```

Se captura aquí

Se propaga

Se lanza aquí

Con esto, el código que invoque al método division, sabe que tiene que **capturar** la excepción **o** bien seguir **propagándola**.

¿propagación o captura?

- Toda excepción de tipo checked debe ser capturada o propagada

¿Cómo saber si se debe capturar o propagar?

- Si podemos resolverla o hacer algo útil → capturarla
- En otro caso → propagarla

Ejercicio 2

En tu programa de la calculadora, o en el ejercicio anterior, crea un método que realice la división y que sea invocado desde otro método (por ejemplo, desde el método main).

Añade el código necesario para propagar las excepciones que puedan producirse en el método de división.

Captúralas en el método que invoca al método de división (por ejemplo, en el main)

Lanzamiento

Java lanza automáticamente (implícitamente) las excepciones.

Pero podemos también lanzarlas manualmente (explícitamente)

palabra reservada: **throw**

Ejemplo:

```
if (x == null)
    throw new IllegalArgumentException("No se admiten nulos");
```

Ejercicio 3

Lanzar una excepción manualmente cuando se intenta meter una edad < 0 a una persona

Persona.java

```java
    public void setEdad(int edad) {
        if (edad < 0) {
            throw new IllegalArgumentException("Edad debe ser 0 o mayor");
        }
        this.edad = edad;
    }

    public Genero getGenero() {
```

EjemploPersona.java

```java
    public static void main(String[] args) {
        Persona fran = new Persona();
        fran.setEdad(-4);
        System.out.println("La edad es: " + fran.getEdad());
    }
```

Relanzar

Una excepción puede ser relanzada después de ser capturada.

Ej: hacemos un tratamiento intermedio pero queremos que se siga propagando

```
try {
    // código susceptible de generar excepción
} catch (ClassNotFoundException ex) {
    // código que controla la excepción
    throw ex; // también podría lanzar una nueva de otro tipo
}
```

Ejercicio 4

Realiza un método que introduzca números enteros en una lista. Invoca este método desde un main.

A continuación programa que se lance una excepción cuando al método se le pasa un número que ya está contenido en la lista. Deberá construirse la excepción con un mensaje descriptivo.

Después, adapta la firma del método para indicar que este método puede propagar la excepción que hemos lanzado.

Finalmente, captura la excepción dentro del main, imprimiendo la traza y el mensaje que lleva dicha excepción.

Excepciones personalizadas

Podemos crear nuestras propias excepciones *ad-hoc* y así usar excepciones específicas a cada programa.

Para ello, creamos una nueva clase que debe extender de la clase de excepción adecuada. Si extiende de alguna de RuntimeException será unchecked y si lo hace de alguna de Exception será checked.

```
public class ExcepcionEdadInvalida extends IllegalArgumentException {
    public ExcepcionEdadInvalida(String mensaje) {
        super(mensaje);
    }
}
```

Ejercicio 5

Crea una excepción llamada NumeroRepetido para el ejercicio anterior.

Pasos:

1º. Crear clase NumeroRepetido que herede de IllegalArgumentException

2º. Crear constructor con parámetros que reutilice el código del padre

3º. Lanzar la excepción creada en lugar de IllegalArgumentException en el ejercicio anterior

Buenas prácticas

- Nunca se debe ocultar una excepción, es decir, capturarla y no hacer nada en el código del catch, ya que los errores pasarán inadvertidos.
- Por regla general, una excepción se debe capturar lo antes posible, para así recomponer la ejecución del programa lo antes posible.
- Liberar recursos mediante bloque finally o try-with-resources
- Evitar usar excepciones genéricas, pues aportan poca información. Hay que ser lo más preciso posible.
- Evitar excepciones personalizadas, se reutiliza código y es más sencillo de interpretar para otros desarrolladores.
- No abusar de excepciones en pro de un código bien diseñado.

Aserciones

Para depurar el código mientras desarrollamos

```
// Asegurar que una variable sea positiva
int x = -15;
assert x > 0 : "El valor debe ser positivo";
System.out.println("Valor positivo x: " + x);
```

```
java.lang.AssertionError: El valor debe ser positivo
```

No deben usarse en producción, solo durante el desarrollo o pruebas.

Hay que ejecutar con el argumento **-ea**

Ejercicios

Ejercicio 6 – Haz una clase que defina las variables a=0, b=300 y calcula el cociente de b/a. Controla el error que da, dando un mensaje de «Se ha producido un error». Al final del programa, SIEMPRE deberá indicar «Programa finalizado»

Ejercicio 7 – Modifica el programa anterior, para que en vez de procesar el error, lo pase hacia el módulo llamador (en este caso la máquina virtual)

Ejercicio 8 – Modifica el programa anterior para que, cuando se produzca el error, en lugar de imprimir por consola el mensaje «Se ha producido un error», lo lance como excepción para que lo capture la clase llamadora (la JVM).

Ejercicio de ampliación

Completa el proyecto de la calculadora con todo lo que has aprendido en esta unidad

Por ejemplo, evita las salidas incontroladas del programa cuando se produce un error de división por cero, la entrada de valores no numéricos, etc.

Puedes añadir excepciones personalizadas para algunos casos.

Contenido de ampliación

Investiga try-with-resources

Cuestionario de autoevaluación

Aplica a tu proyecto

1. Crear grupos de trabajo - 5’
2. Pensar propuesta de proyecto - 10’
3. Redactar propuesta en un documento de texto - 10’
4. Crear repositorio, readme.md y copiar link en la tarea - 5’
5. Elaborar prototipo (papel o digital) y casos de uso - 20’
6. Validar propuesta con el profesor conforme vayáis acabando
7. Crear diagrama de clases (versión inicial) - 20’
8. Crear las clases y el main en Java - 30’
9. Aplicar lo que hemos estudiado en la unidad (Excepciones) - 30’